

Algoritmos Paralelos



Mestrado em Computação Avançada

João L. Sobral
jls@di.uminho.pt

Web: Elearning

Parallel Algorithms

This course aims to strengthen skills in the area of **parallel algorithms** in problem solving. The main objective is to show, through concrete applications ("case-studies") that require algorithms of high computational complexity, the importance of selecting/developing appropriate [parallel] algorithms.

Learning Outcomes

A student who successfully completes this course must be able to demonstrate that he or she has acquired the following skills:

- Know the main parallel algorithms and know how to select a suitable parallel algorithm for a given problem/target platform
- Identify, at different levels (complexity, robustness, efficiency, scalability, etc.) strengths and weaknesses of the algorithms analyzed
- Develop sequential and parallel implementations and discuss the performance and efficiency results obtained with the analyzed algorithms
- Identify possible load balancing issues and/or high communication costs in parallel algorithms

Parallel Algorithms

Detailed Program

- Models & metrics for scalability analysis
- Basic algorithms: map/reduce/scan; broadcast/scather/gather
- Monte Carlo methods: sequential and parallel random number generators
- Matrix algorithms: matrix multiplication, systems of linear equations
- Parallel algorithms for scientific computing: molecular dynamics/N-Body, AI;
- Sorting algorithms: mergesort, quicksort; parallel sort by regular sampling
- Algorithms on irregular data structures: graphs and trees
- Fine grained parallelism: lock-free algorithms
- Speculative parallelism
- JIT compilers & Managed languages

Note: no technology here!
We will try to understand
the algorithm essence
& concretise with:
OpenMP | MPI | CUDA

Parallel Algorithms

Evaluation: Three practical assignments & test (25% of grade each)

Bibliografia: slides & On-line material

Schedule (tentative)

1. Models & basic algorithms (4/fev)
2. MMult: BLAS1 - 3 (11/Fev)
3. **BLAS1 a 3 & Tensor core (18/fev)**
4. Particle Methods & N-body... (25/fev)
5. Sorting (4/mar)
6. Stencil (11/mar)
7. IA (18/mar)
8. Tree / Graphs (25/mar)
9. **Páscoa 30/mar a 6/abr**
10. Lock free (8/abr)
11. Speculative(15/abr)
12. Others (22/abr)
13. Managed & JIT (28/abr)
14. **Folga ou teste (6/maio)**
15. Enterro da Gata

Parallel Algorithms

□ Traditional algorithm analysis: Big O notation

■ Analysis of the number of operations per datum (n)

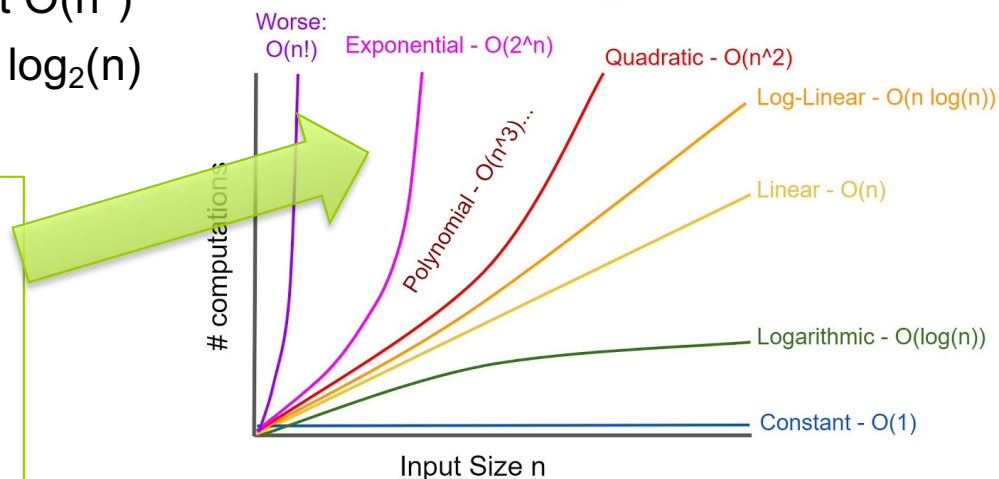
- Matriz multiplication: $\Theta(n^3)$ $O(n^{\frac{3}{2}})$ (n=number of elements of C)
- Sorting (n keys)
 - [almost] Worst sorting (burte force): $O(n^2)$
 - Better algorithm: $n \log_2(n)$
 - Best: $([k]n)$
- Insertion on a data structure (n insertions):
 - Sorted linked list $O(n^2)$
 - Binary tree: $O(n \log_2(n))$
 - Hash table $O(n)$

Traditionally these are the best for parallel computing

Why?

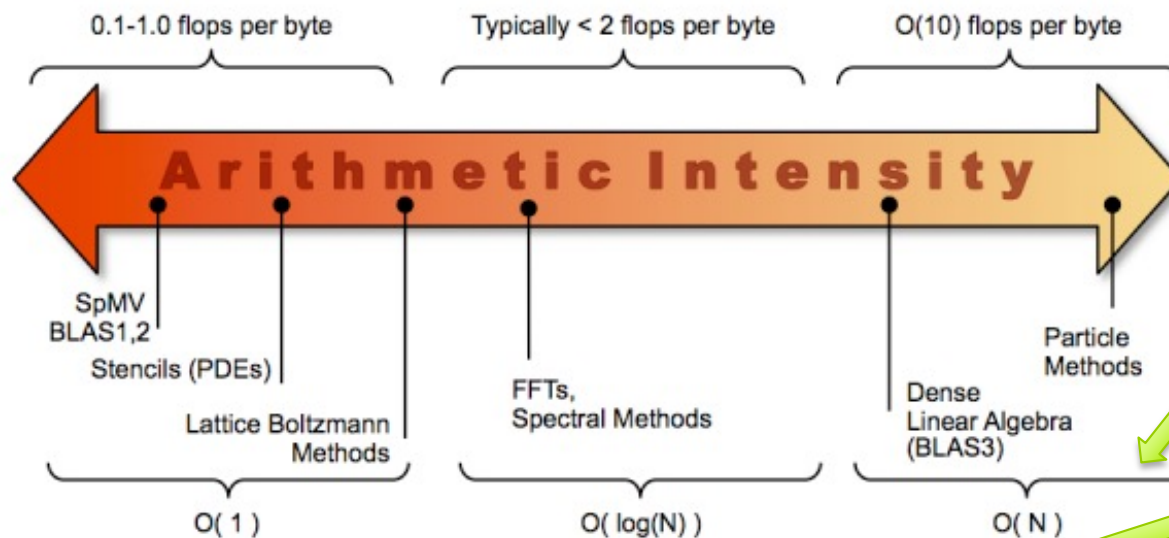
Too complex for sequential machines (years2solve!)

Sugg. : 10000x10000 MMult



Parallel Algorithms

□ Arithmetic Intensity (operations per ~~byte loaded~~^{data moves})



Data centric-analysis

How many times each datum can be reused?

What is the implication of this on algorithm scalability?

□ Data storage can be

- Regular : vector, matrix
- Irregular (pointer based): tree, graphs

Data accesses can be

- Regular / irregular

Conclusion: [parallel] algorithms with low arithmetic intensity (& irregular data storage/data accesses) will generally not scale well

Parallel Algorithms

□ Performance metrics & models

■ Speed-up (strong scaling) & Efficiency

■ Weak scaling - problem size increases as the number of processes/threads increases

■ Roofline model

- Bandwidth based analysis

■ External memory model

- Latency based model

Complementary analysis

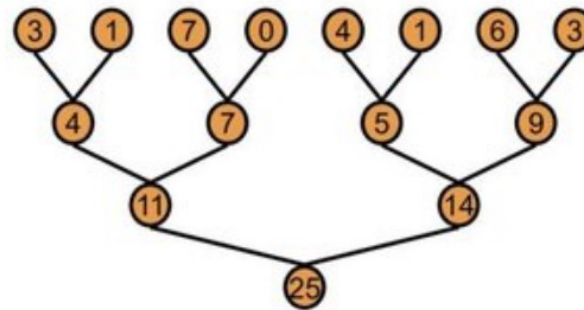
Memory bandwidth vs I/O operations (memory latency)

- Memory is organized into internal (cache) & external memory (main memory)
- The goal is to account for the number of data blocks transferred between memory levels

Parallel Algorithms

□ Algorithmic problems for communication

- **Reduce ($\downarrow$) Broadcast ($\uparrow$)**
all are implemented
with ONE parallel tree
- **Also these:**
 - Scatter/gather
 - All-to-all ???
 - Allreduce = reduce + broadcast?
 - Allgather = gather + broadcast?

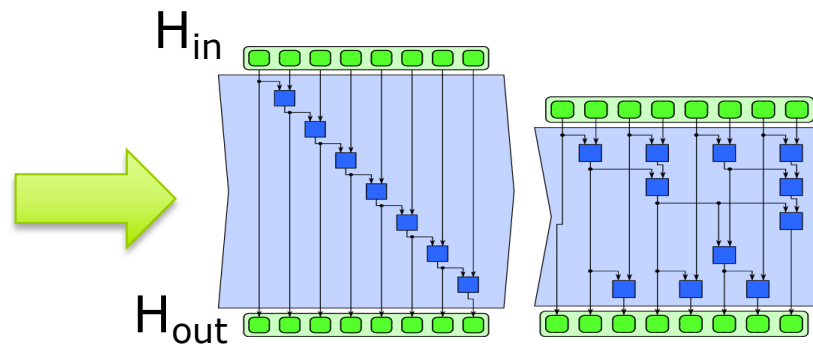


A parallel
reduction
requires
 $\log_2 n$
parallel
steps

The idea can be applied to many
other cases (barrier, thread creation)

■ Parallel scan

```
void operation_2() {  
    for (int n = 1; n < Nc; n++) {  
        H[n] += H[n-1];  
    }  
}
```

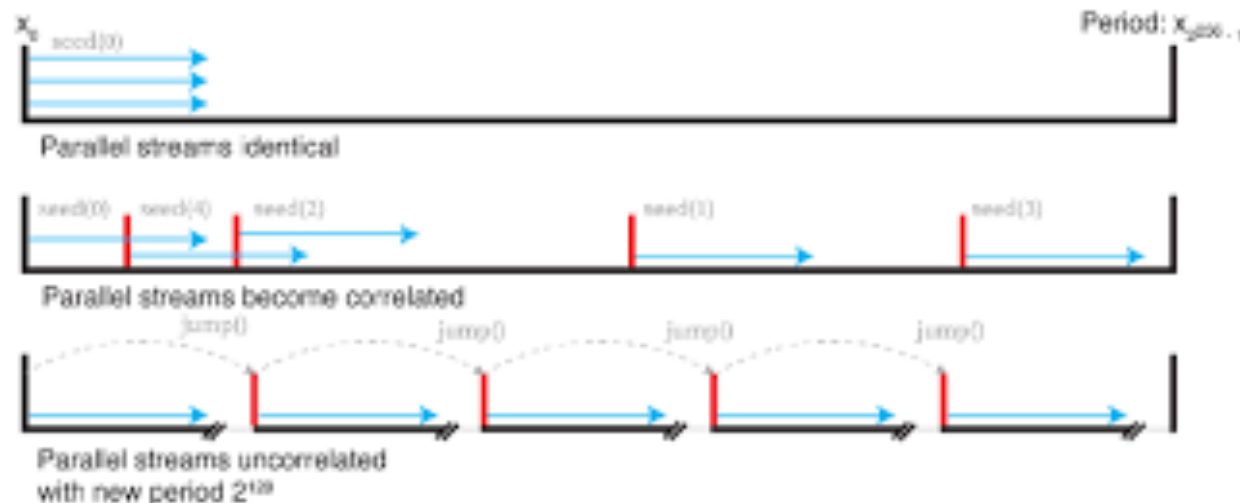


Sequential
scan

parallel scan (based on
a parallel reduction)

Monte Carlo Methods

- ❑ Broad class of computational algorithms that use repeated random sampling to obtain numerical results for complex problems
 - **usual bottleneck is the random number generation**
- ❑ Parallel random number generation
 - Random sequences must be uncorrelated



Best approach:
- Use a larger period (e.g. 48bits) & disjoint sequences for each thread

Practical Lab 1 – Monte Carlo Pi computation

1. Profile the provided code for pi computation:

```
for (int i = 0; i < (INTERVAL * INTERVAL); i++) {  
    double rand_x = double(rand() % (INTERVAL + 1)) / INTERVAL;  
    double rand_y = double(rand() % (INTERVAL + 1)) / INTERVAL;  
    double origin_dist = rand_x * rand_x + rand_y * rand_y;  
    if (origin_dist <= 1)    circle_points++;  
    square_points++;  
}  
cout << "\nFinal Estimation of Pi = " << double(4 * circle_points) / square_points;
```

2. Develop & profile a simple OpenMP parallel version

3. Develop a version using `rand_r(unsigned int *seedp)`

4. Explore alternatives (e.g., the `erand48` / CUDA)